

NOTICE TECHNIQUE ET DOCUMENTATION DÉVELOPPEUR

Suite Logicielle SeqXplorer - CHU de Liège

Architecture, Maintenance et Développement

Cette notice technique s'adresse aux administrateurs systèmes, bioinformaticiens ou développeurs reprenant la gestion de l'application SeqXplorer. Elle détaille l'architecture logicielle, les mécanismes de sécurité, la logique algorithmique des pipelines et les procédures de maintenance.

Introduction & Philosophie de l'Application

SeqXplorer a été conçue pour automatiser et standardiser les routines de diagnostic génétique (Bactériologie 16S, Virologie HCV, Mycologie). La stack technologique privilégie la robustesse et l'autonomie locale, en s'appuyant sur :

- **Python 3.13** comme moteur principal.
- **Streamlit** pour une interface utilisateur web locale, réactive et sans déploiement serveur.
- **Biopython** pour le traitement standardisé des séquences (Sanger .ab1, FASTA, alignements).
- **Playwright** pour l'automatisation des requêtes sur les portails web experts dépourvus d'API.

PARTIE 1 : Architecture & Environnement

1.1 Arborescence globale

L'application est structurée de manière modulaire pour séparer le code source, les binaires et les résultats cliniques :

- **app_SeqXplorer/** : Contient le cœur du code source, les scripts Python et l'environnement virtuel.
- **bin/** : Dossier hébergeant les exécutables locaux (notamment **blastn.exe** et ses DLL).
- **databases/** : Héberge les bases de données locales (ex: base curatée Emu pour le 16S).
- **Notices/** : Stockage des documentations utilisateur (PDF).
- **Résultats_SeqXplorer/** : Dossier racine généré dynamiquement en dehors de l'application, centralisant tous les outputs cliniques pour faciliter l'archivage.

1.2 L'environnement virtuel (venv)

Pour éviter tout conflit avec le système d'exploitation de l'hôpital, l'application tourne dans un environnement virtuel strict. Le fichier batch **Installation (Ne lancer qu'une fois).bat** est chargé de créer ce dossier **venv** et d'y installer les paquets. En cas de corruption, il suffit de supprimer le dossier **venv** manuellement et de relancer le script d'installation.

1.3 Gestion des dépendances (**requirements.txt**)

Les paquets fondamentaux de l'application sont :

- **streamlit** : Génération de l'interface graphique.
- **biopython** : Parsing des fichiers **.ab1**, algorithmique de séquençage.
- **playwright** : Scraping et pilotage de navigateurs virtuels (Headless).
- **reportlab** & **pypdf** : Dessin dynamique et fusion des rapports cliniques PDF.

1.4 Outils externes & Binaires locaux

SeqXplorer embarque **blastn.exe** pour une exécution locale sans dépendance réseau.

Note de sécurité (EDR) : Les environnements hospitaliers bloquent souvent les exécutables non signés lancés via des scripts. L'application est programmée pour capturer silencieusement les crashes de type *Access Violation* (dus à l'antivirus ou à l'absence de bibliothèques C++) et basculer sur des protocoles de secours (API web).

PARTIE 2 : Mécanique de l'Interface Utilisateur (UI)

2.1 Le cycle de vie Streamlit

Streamlit recharge l'intégralité du script de haut en bas à chaque interaction de l'utilisateur (clic, saisie). Pour éviter que le pipeline bioinformatique ne se relance en boucle, l'application repose massivement sur la gestion d'état.

2.2 Gestion de l'état (**st.session_state**)

Lors du lancement d'un run, l'UI est verrouillée grâce aux variables de session (**is_running**, **analyze_clicked**). Ces booléens sont injectés dans l'attribut **disabled** des champs de texte et des zones de dépôt, empêchant l'utilisateur de corrompre les données d'entrée pendant que la boucle d'analyse tourne.

2.3 Le mécanisme d'interruption d'urgence (Bouton STOP)

Le bouton Stop utilise un **on_click** callback. Lorsqu'il est déclenché :

1. Les variables d'état de fonctionnement sont réinitialisées (**False**).

2. Le module `shutil.rmtree` cible les dossiers temporaires (`Temp_Inputs`) et le dossier du patient exact qui était *en cours de traitement* au moment du clic, garantissant qu'aucun fichier consensus partiel ou rapport corrompu ne soit sauvegardé.

PARTIE 3 : Le Cœur Bioinformatique (Pipelines)

3.1 Algorithmique d'appairage Regex

La fonction `scan_directory` apparie les brins Forward et Reverse. L'expression régulière est conçue pour être agnostique : elle nettoie les préfixes de laboratoires variables (ex: `MIC-dermato_`) et cible la suite d'identifiants se terminant strictement par `f` ou `r` juste avant l'extension `.ab1`.

3.2 Moteur de Qualité (QC) et Trimming

Chaque chromatogramme subit une évaluation stricte :

- Calcul du pourcentage de bases Q20 et de la moyenne Phred globale.
- **Trimming** : Le script balaie la séquence depuis les extrémités par blocs (`window_size=5`). Si la moyenne Phred du bloc chute sous 10, la séquence est coupée. Cela permet de retirer le bruit de fond typique des lectures Sanger.

3.3 Assemblage du Consensus

L'alignement local utilise `Align.PairwiseAligner`.

- Le brin Reverse est automatiquement transformé en *Reverse Complement*.
- **Résolution des conflits** : En cas de mismatch (ex: `A` sur le brin F, `T` sur le brin R), l'algorithme compare la qualité Phred des deux bases. La base est conservée si son score surpasse l'autre d'au moins 5 points. Dans le cas contraire, une base ambiguë `N` est insérée (mécanisme basé sur l'analyse manuelle sur geneious primer réalisée avant la mise en place de SeqXplorer).

PARTIE 4 : Modules de Taxonomie et d'Interrogation

4.1 Module 16S (BLAST Local & API NCBI)

Le module tente d'abord une identification locale Emu via `subprocess.run()`.

- **Tolérance aux pannes (Fallback)** : Si `shutil.which` ne trouve pas d'installation de BLAST, ou si l'EDR tue le processus local, l'exception est interceptée (warning) et l'algorithme bascule automatiquement sur `NCBIWWW.qblast`, interrogeant de façon synchrone les bases `16S_ribosomal_RNA` puis `nt` (réalisé systématiquement).

4.2 Module HCV (Scraping Geno2Pheno)

Ce module utilise Playwright en mode Headless. Le robot charge le portail, injecte le FASTA dans l'élément HTML `<textarea>`, soumet la requête et attend la résolution réseau. Le génotype et l'indice de confiance sont extraits du texte source par Regex, et le rapport natif certifié du serveur est intercepté via la fonction `page.expect_download()`.

4.3 Module Champignons (Scraping KNAW)

Le portail Westerdijk Institute étant particulièrement lent et dynamique, ce module intègre une boucle d'attente complexe :

- **Stable Checks** : L'automate scrute le DOM toutes les 2 secondes (`page.locator("tr")`). Il ne considère la page comme chargée que si le nombre de lignes du tableau reste identique durant 5 itérations consécutives, avec un garde-fou (Timeout) fixé à 160 secondes.

PARTIE 5 : Traçabilité, Logs et Génération de Rapports

5.1 Système de Logging avancé

Chaque patient dispose de son propre objet `logger` (`logging.FileHandler`) qui génère un fichier `pipeline_[ID_Patient].log`. Ce fichier est la "boîte noire" de l'analyse, capturant :

- Les jalons du pipeline (nettoyage, requêtes).
- Les métriques de qualité (scores Q20, taux de chevauchement, conflits résolus).
- Les erreurs systèmes (Crash silencieux d'un sous-processus Windows, Timeout Playwright).
- Les Stack Traces Python natives (via `traceback.format_exc()`).

5.2 Moteur PDF (`reportlab` & `pypdf`)

Les rapports sont dessinés programmatiquement par `reportlab` (mise en page, police, tableaux récapitulatifs des "Hits" avec coloration syntaxique des alertes). Dans des modules comme le HCV, `pypdf` est ensuite utilisé pour fusionner cette page de garde générée localement avec le PDF technique original récupéré par l'automate.

PARTIE 6 : Maintenance, Mise à jour & Évolutivité

6.1 Le système de Mise à jour (MaJ)

L'application est conçue pour être mise à jour automatiquement. Le script de lancement `SeqXplorer.bat` inclut une routine de vérification. A chaque lancement, le programme va scanner le repertoire github du projet (<https://github.com/JeremiePln/SeqXplorer>) et detecter

dans le dossier updates si une nouvelle update est recensée dans updates_list.txt. Ensuite, il va automatiquement télécharger l'update avec le bon index (updateXX.zip) et fusionner tout le dossier update au dossier SeqXplorer déjà initialisé via la commande **xcopy**. Les dossiers d'update doivent respecter l'arborescence de SeqXplorer parfaitement, en considérant le dossier updateXX/ comme le dossier SeqXplorer/. Si on veut donc mettre à jour un script 16s on construit l'update de la sorte :
update_00/app_SeqXplorer/scripts/app_taxonomie_16s/app_16s.py (avec la nouvelle version de app_16s.py), on zip ensuite ce dossier update00/ et on l'implante dans le github + on l'indexe dans la liste des mise à jour (si c'est la .7eme, on l'appelle update_07 et on ajoute dans le txt 7:update_07.zip).

Une fois une mise à jour lancée, son index est enregistré localement dans historique_maj.txt pour que celle-ci ne soit pas relancée.

Note : jeremie.poulain@estbb.ucl.ac.uk ou jeremiepoulain0110@gmail.com pour demander l'accès au github

6.2 Ajouter un nouveau module (Évolutivité - V4)

Pour intégrer un futur 4ème module (ex: Mycobactéries) :

1. Créer un nouveau sous-dossier **app_[nom_module]** dans le répertoire **scripts/**.
2. Dupliquer les fichiers de base d'un module existant (UI verrouillable, pipeline QC, robot web, éditeur PDF), ou partir de fichiers vierges.
3. Déclarer le nouveau bouton de navigation dans le fichier partagé de la barre latérale **styles.py**.
4. Lier les nouvelles pages dans le menu de diagnostic (page d'accueil du module).